

CO4-AT-1

Comparative Analysis

Name:- R. JANANI SRI

Course:- Data Structures

Code:- CSA0307

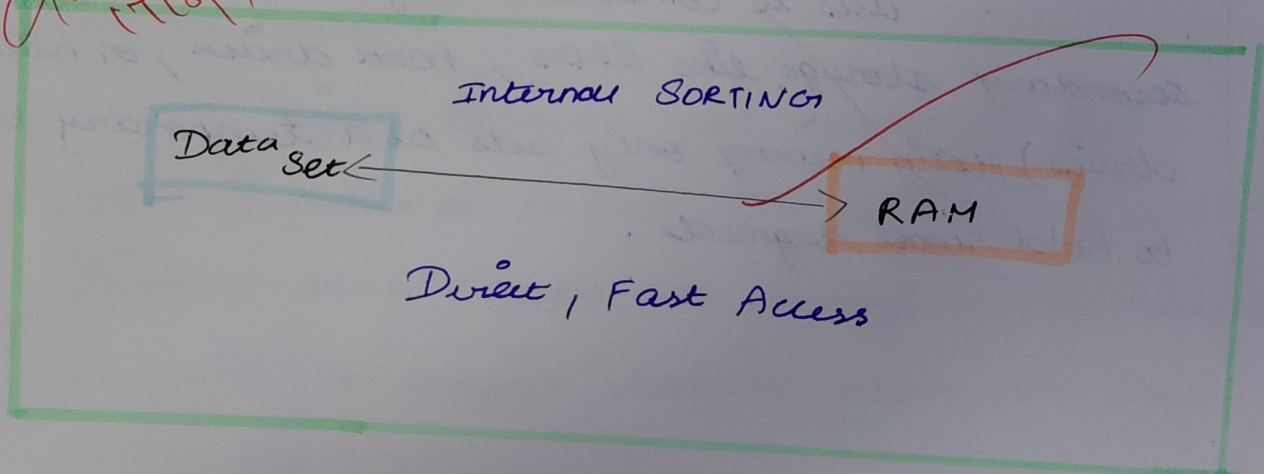
Reg no:- 192511093

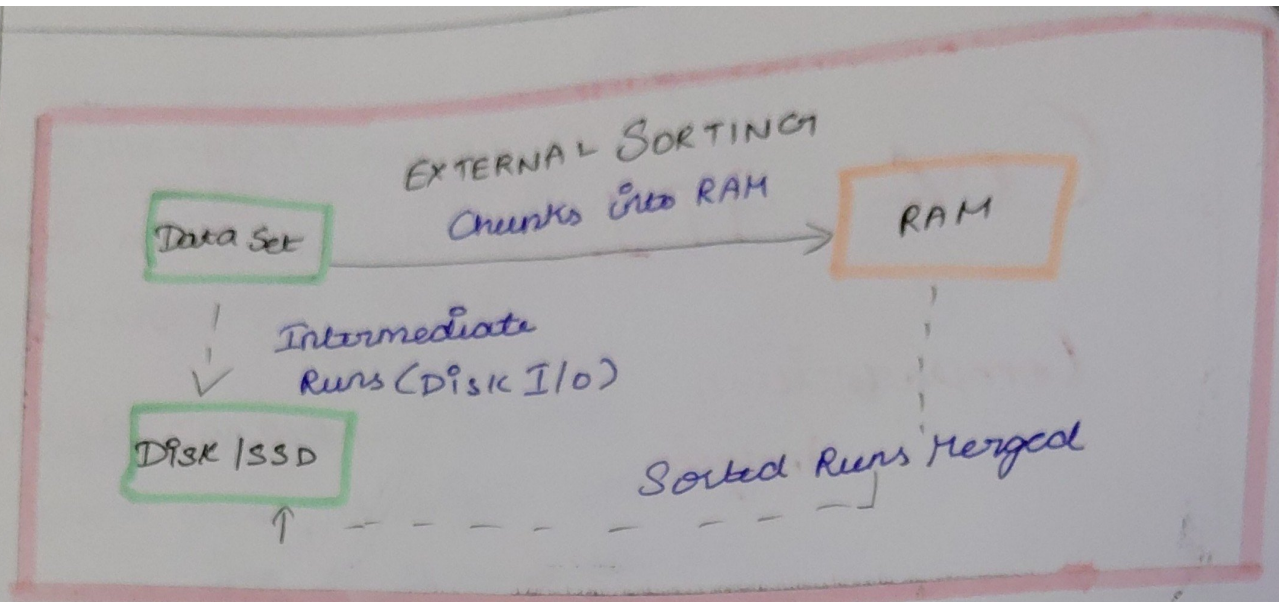
Compare heap-based priority queue and array-based
Priority queue in terms of

Internal Sorting Vs External Sorting

The primary dividing line between internal and external sorting is where the data resides during the sorting process. This choice dictates how the algorithm accesses records and directly influences the system's architecture

Ans
17/8/22





a.) Memory usage:-

Internal Sorting:-

Runs entirely within the Computer's Volatile main memory (RAM). The algorithm requires enough RAM to store the collection all at once, alongside a tiny allocation for variable adjustments (in place) or an array copy.

External Sorting:-

uses a combination of main memory and secondary storage like SSDs, hard drives, or network drives. Main memory only acts as a temporary buffer to hold small segments.

b) Data size handled:-

Internal Sorting:-

Limited by physical hardware boundaries. It is designed exclusively for small to medium-sized datasets that fit comfortably within the available RAM allocation. If the dataset size grows too large, the system throws out of memory errors.

External Sorting:-

Scalable up to terabytes or petabytes of data. It accommodates massive datasets like enterprise database registries, global transactions that are vastly larger than the computer's physical memory capacity.

c) Performance Considerations:-

Internal Sorting:-

Highly efficient and extremely fast because it relies on RAM. The primary performance indicators are CPU cycles and memory word-swapping operations. It benefits heavily from CPU cache optimization.

External Sorting:-

Bound by secondary storage bottlenecks
Because physical storage media are significantly slower than RAM, performance depends on reducing Disk I/O operations

Parallel Merge Sort vs Sequential Merge Sort

Feature Comparison	Sequential Merge Sort	Parallel Merge Sort
Primary Resource	Single CPU thread / core	Multiple CPU Cores / threads
Execution Path	Single, linear stack	Forked, tree like worker thread pool
Algorithmic Overhead	Near zero (only standard execution stack)	High (Thread management)
Resource Utilization	Low (Leaves other CPU cores idle)	High (Makes use of multi core)
Data Suitability	Small to medium array	Massive, highly divisible

a) Execution time:-

Sequential Merge Sort:-

Processes partitions one after the other on a single processor core. For an array of size N , it follows a strict $O(N \log N)$ execution curve.

Parallel Merge Sort:-

Drastically drops overall execution wall-clock time on multiple hardware. By assigning separate array branches to independent threads, it completes multiple sub-sorting tasks concurrently.

b.) Overhead due to parallelism:-

Sequential Merge Sort:-

Suffers no extra management costs. Every byte of computational power directly pushes the array closer to its sorted state.

Parallel Merge Sort:-

Faces significant hidden performance penalties. The system must spend processing power creating, scheduling and destroying threads.

If data chunks are too small, this management overhead can take longer than the sort itself.

C) Suitability for Large Datasets:-

Sequential Merge Sort:-

Quickly turns into a performance bottleneck when handling large arrays. As data grows, a single CPU core limits the bandwidth to process the exponential depth of the merge tree efficiently.

Parallel Merge Sort:-

Idea for large datasets. Massive arrays provide enough work to keep multiple CPU cores busy, making the parallel management overhead negligible compared to the massive speed gains.